COMPETITIVE PROGRAMMING LAB — MICRO NOTES

1) Find the Unique Element Using XOR — C

```c
#include <stdio.h>
int main() {
    int n, x, unique = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        unique ^= x;
    }
    printf("%d\n", unique);
    return 0;
}
```

2) Reverse a 32-bit Integer — C

```c
#include <stdio.h>
#include <limits.h>
int reverseInteger(int n) {
    int rev = 0, digit;
    while (n != 0) {
        digit = n % 10;
        n /= 10;
        if (rev > INT_MAX / 10 ||
            (rev == INT_MAX / 10 && digit > 7))
            return 0;
        if (rev < INT_MIN / 10 ||
            (rev == INT_MIN / 10 && digit < -8))
            return 0;
        rev = rev * 10 + digit;
    }
    return rev;
}
int main() {
    int n;
    scanf("%d", &n);
    printf("%d\n", reverseInteger(n));
    return 0;
}
```

3) Fast Exponentiation Using Tail Recursion — C

```c
#include <stdio.h>
double power_tail(double x, unsigned long long n, double acc) {
    if (n == 0) return acc;
    if (n % 2 != 0) acc *= x;
    return power_tail(x * x, n / 2, acc);
}
double my_pow(double x, long long n) {
    unsigned long long exp;
    if (x == 0 && n < 0) return 0;
    if (n < 0) {
        exp = (unsigned long long)(-(n + 1)) + 1;
        return power_tail(1.0 / x, exp, 1.0);
    }
    exp = (unsigned long long)n;
    return power_tail(x, exp, 1.0);
}
int main() {
    double x;
    long long n;
    scanf("%lf %lld", &x, &n);
    printf("%.5f\n", my_pow(x, n));
    return 0;
}
```

4) Generate All Unique Subsets (Power Set) — Python 3

```python
def subsets(a, i=0, path=None):
    if path is None:
        path = []
    if i == len(a):
        print(path)
        return
    subsets(a, i + 1, path + [a[i]])
    subsets(a, i + 1, path)

a = list(map(int, input().split()))
subsets(a)
```

5) Rotate a Matrix 90 Degrees Clockwise — Python 3

```python
n = int(input())
a = [list(map(int, input().split())) for _ in range(n)]
a = [list(row) for row in zip(*a)]
for row in a:
    row.reverse()
for row in a:
    print(*row)
```

6) String Compression — Python 3

```python
s = input()
result = ""
i = 0
while i < len(s):
    count = 1
    while i + count < len(s) and s[i] == s[i + count]:
        count += 1
    result += s[i] + str(count)
    i += count
if len(result) >= len(s):
    print(s)
else:
    print(result)
```

7) Student Records Using Binary Files — Python 3

```python
import pickle
n = int(input())
students = []
for _ in range(n):
    id, name, gpa = input().split()
    students.append((int(id), name, float(gpa)))
with open("records.db", "wb") as f:
    pickle.dump(students, f)
with open("records.db", "rb") as f:
    data = pickle.load(f)
for id, name, gpa in data:
    if gpa >= 8.5:
        print(f"ID: {id} | Name: {name} | GPA: {gpa:.2f}")
```

8) Tagged Union for Sensor Readings — Python 3

```python
while True:
    try:
        line = input().strip()
        if line == "-1":
            break
        t, v = line.split(maxsplit=1)
        if t == "1":
            print(f"[TEMP] {float(v):.2f} C")
        elif t == "2":
            print(f"[STATUS] {v}")
        elif t == "3":
            print(f"[COUNT] {int(v)}")
    except EOFError:
        break
```

9) Reverse a Singly Linked List in Groups of K — Python 3

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def reverse_k(head, k):
    if k <= 0:
        return head
    dummy = Node(0)
    dummy.next = head
    prev = dummy
    while True:
        check = prev
        for _ in range(k):
            check = check.next
            if not check:
                return dummy.next
        curr = prev.next
        tail = curr
        p = check.next
        for _ in range(k):
            nxt = curr.next
            curr.next = p
            p = curr
            curr = nxt
        prev.next = p
        prev = tail

n, k = map(int, input().split())
a = list(map(int, input().split()))
head = tail = None
for x in a:
    node = Node(x)
    if head is None:
        head = node
    else:
        tail.next = node
    tail = node
head = reverse_k(head, k)
while head:
    print(head.data, end=" ")
    head = head.next
print()
```

10) Rotate a Doubly Linked List Counter-Clockwise — Python 3

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

n, p = map(int, input().split())
a = list(map(int, input().split()))
head = tail = None
for x in a:
    node = Node(x)
    if head is None:
        head = node
    else:
        tail.next = node
        node.prev = tail
    tail = node
if n:
    p %= n
if head and p:
    temp = head
    for _ in range(p - 1):
        temp = temp.next
    new_head = temp.next
    tail.next = head
    head.prev = tail
    temp.next = None
    new_head.prev = None
    head = new_head
while head:
    print(head.data, end=" ")
    head = head.next
print()
```